

FAIR SHARE

Architecture and Capabilities of a Multi-Client Expense Splitter

Sotirios Goulas

September 2026

Technical report on the `fair-share` repository, version 0.1.0

Abstract

FAIR SHARE is a small system for splitting trip expenses among a group of people without floating-point money. Amounts are stored and computed as integer cents. Equal shares use integer division with a deterministic remainder; weighted shares use the Hamilton (largest-remainder) method. A greedy pairing of largest debtor and largest creditor produces a settlement. The repository contains three clients that speak the same trip model: a Python command-line tool with atomic JSON persistence, a vanilla TypeScript progressive web app, and an Expo application for iOS, Android, and a web preview. Hosted trips live behind Netlify Functions, with display JPEGs in Netlify Blobs and optional full-resolution originals in Cloudinary. Anyone who has a trip link may edit people and expenses. A six-digit PIN, shown once at trip creation, gates the photo gallery. This report describes the repository layout, the money algorithms, the trust model, and the capabilities of each client.

Contents

- 1 Purpose
- 2 Repository layout
- 3 Domain model
- 4 Algorithms
 - 4.1 Equal split · 4.2 Weighted split · 4.3 Settlement · 4.4 Persistence
- 5 Clients
 - 5.1 Command-line interface · 5.2 Web application · 5.3 Mobile application
- 6 Hosted service
- 7 Trust and security model
- 8 Capabilities, by concern
- 9 Development, tests, and deployment
- 10 Limits and non-goals
- 11 Licence
- 12 Summary

Production site: <https://fair-share-trips.netlify.app>

1 Purpose

Splitting a shared bill looks simple until a remainder cent appears, a hotel is paid by one person and used unequally, or four people try to edit the same list from four phones. FAIR SHARE treats those as the product, not as afterthoughts.

The system does four things:

1. Record who paid, how much, and who took part, using only integer cents.
2. Compute net balances and a short list of repayments.
3. Let a group share one trip over a URL, without accounts.
4. Attach a PIN-gated photo gallery (“Moments”) to a hosted trip, with automatic expiry after one year.

It does not provide user accounts, real-time collaboration locks, receipts as financial records, or in-app payments. The mobile client may show an optional external donation link; that link is ordinary HTTPS in the system browser, not a store purchase.

2 Repository layout

The Git repository is a monorepo. Money rules exist in two implementations that are intended to agree: Python under `src/fairshare/` for the CLI, and TypeScript under `packages/domain/` for the hosted stack.

Table 1. Top-level packages and their roles.

Path	Role
<code>src/fairshare/</code>	Python CLI: trip file, splits, settlement, pull/push
<code>tests/</code>	Pytest suite for the CLI (coverage gate 80%)
<code>packages/domain/</code>	Shared TypeScript money, PIN, and photo constants
<code>web/</code>	Vite + TypeScript PWA (euro UI, trip links, Moments)
<code>netlify/functions/</code>	HTTP API: trips, photos, signed originals, expiry
<code>mobile/</code>	Expo SDK 57 client (same HTTP API; EAS for store builds)
<code>netlify.toml</code>	Build, functions directory, SPA rewrite for <code>/t/*</code>

The web bundler and Vitest resolve `@fairshare/domain` to `packages/domain/src`. Netlify Functions import the same sources by relative path and are typechecked with `netlify/tsconfig.json` during the production build. The Expo app consumes the hosted JSON over HTTP; it re-exports a thin `mobile/src/domain` boundary and must not import server-only PIN hashing or Cloudinary signing into that barrel.

Native `ios/` and `android/` trees are not in the repository. Store binaries are produced with EAS in the cloud. The application identifier is `app.fairshare.trips`; the URL scheme is `fairshare`.

3 Domain model

A trip is a named collection of people and expenses. Schema version is currently 1. Hosted trip identifiers are 32 lowercase hexadecimal characters (16 random bytes).

An expense records a description, a payer who must already be on the trip, a positive amount in cents, a non-empty list of distinct participants, and an optional weight vector of the same length. Person matching is case-insensitive; the first spelling seen is the canonical one. Comma characters are forbidden in names so that CLI lists stay unambiguous.

Balances are defined in the obvious way. If person p paid an expense of A cents, p 's balance increases by A . Each participant i is then charged share s_i , and $\sum_i s_i = A$ always holds because shares are computed in integer cents.

$$B(p) = \sum_{e \in E, \text{payer}(e) = p} A_e - \sum_{e \in E, p \in P_e} s_e(p) \quad (1)$$

A positive balance means the group owes that person money. A negative balance means that person owes the group.

4 Algorithms

4.1 Equal split

For an amount $A \in \mathbb{Z}_{>0}$ and $n \geq 1$ participants,

$$s_i = \left\lfloor \frac{A}{n} \right\rfloor + 1 \left[i < (A \bmod n) \right], \quad i = 0, \dots, n-1 \quad (2)$$

The first $A \bmod n$ people receive one extra cent. The order is the participant list as stored, so the result is deterministic and the shares sum to A .

4.2 Weighted split

Weights w_i are positive integers. Let $W = \sum_i w_i$. The Hamilton / largest-remainder method assigns

$$q_i = \left\lfloor \frac{A w_i}{W} \right\rfloor, \quad r_i = (A w_i) \bmod W \quad (3)$$

then distributes the leftover $A - \sum_i q_i$ cents one at a time to the indices with the largest r_i , breaking ties by earlier index. Again $\sum_i s_i = A$, with no binary floating-point in the path.

4.3 Settlement

Let B be the map of non-zero balances. While both a debtor ($B < 0$) and a creditor ($B > 0$) remain, pair the most negative debtor with the most positive creditor (alphabetical name as tie-break) and transfer

$$t = \min(-B(d), B(c)) \quad (4)$$

This is a greedy min-cash-flow heuristic. Finding a globally minimal number of transactions is NP-hard; the heuristic is used because it is simple and tends to produce a short list in practice.

4.4 Persistence

The CLI writes `fairshare.json` (or a path given by `--file`) by creating a temporary file in the same directory and renaming it into place. A missing or unsupported `schema_version` is a hard error. Hosted trips are JSON documents in a Netlify Blobs store, keyed by trip id. The public GET representation omits the PIN hash.

5 Clients

5.1 Command-line interface

The Python package `fair-share` (Python 3.11+) installs the `fairshare` console script. The default data file is `./fairshare.json`. Amounts accept forms such as `60`, `60.00`, `$60`, and `$60.50`.

Table 2. CLI commands.

Command	Effect
<code>init</code>	Create a trip file; refuses to overwrite unless <code>--force</code>
<code>add-person</code>	Append one or more people
<code>add</code>	Record an equal or weighted expense
<code>edit-expense</code>	Replace an expense by full id or unique prefix
<code>remove-expense</code>	Delete an expense by id or unique prefix
<code>list</code>	Print expenses
<code>balances</code>	Print net balances
<code>settle</code>	Print who pays whom
<code>pull</code>	Download a hosted trip (money only) into the local file
<code>push</code>	Upload the local file as a <i>new</i> hosted trip

`pull` and `push` talk to the same HTTP API as the website. The host is `--host` or the environment variable `FAIRSHARE_API`. The JSON file never contains photos or a PIN hash. `push` always creates a new hosted trip and prints a fresh link and PIN.

5.2 Web application

`web/` is a cream-and-brown progressive web app built with Vite. Currency display is euro. Creating a trip yields a shareable path `/t/<id>`. The home page lists trips seen on that device, creates a new trip, and can **Open JSON** to import people and expenses into a new hosted trip (new id, new PIN).

On a trip page a person can add and remove people, add, edit, and delete expenses, inspect balances and settlement, copy the trip link and the PIN, and **Download JSON** (money only).

Moments is the photo gallery: unlock with the PIN, upload a compressed JPEG, delete a photo, and—when Cloudinary is configured—open a short-lived original URL.

Local development is `npm run dev` on port 8888, which fronts the Vite dev server and the Functions.

5.3 Mobile application

`mobile/` is Expo Router (SDK 57). It opens a trip by id or pasted link, then offers the same money operations and Moments against the production (or configured) API origin, defaulting to <https://fair-share-trips.netlify.app>.

Photo session tokens are stored with Expo SecureStore (an injectable store is used in tests). The root layout waits until that store is ready before rendering the stack, so a cold start does not race a missing token. Relative photo URLs from the API are prefixed with the API base. The Support button appears only when `EXPO_PUBLIC_DONATE_URL` (or `expo.extra.donateUrl`) is a non-empty HTTPS URL.

6 Hosted service

Production is a Netlify site. The build installs `root` and `web` dependencies, typechecks Functions, and publishes `web/dist`. SPA routes `/t/*` rewrite to `index.html`. Response headers include `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, and a strict-origin referrer policy. Functions send CORS headers including `Authorization`, so the Expo web preview can call the API; native `fetch` does not use CORS.

Table 3. HTTP surface (Netlify Functions).

Method	Path	Purpose
POST	<code>/api/trips</code>	Create trip; returns id, public trip, PIN, photo token
GET	<code>/api/trips/:id</code>	Public trip (no PIN hash); <code>photos_locked</code> if a hash exists
PUT	<code>/api/trips/:id</code>	Replace people and expenses (link is authority)
POST	<code>/api/trips/:id/session</code>	Exchange PIN for a photo session token
POST	<code>/api/trips/:id/pin</code>	Set PIN once on a grandfathered trip
GET	<code>/api/trips/:id/photos</code>	List gallery (PIN session if locked)
POST	<code>/api/trips/:id/photos</code>	Upload compressed display JPEG
POST	<code>/api/trips/:id/photos/sign</code>	Signed Cloudinary upload parameters
DELETE	<code>/api/trips/:id/photos/:photoId</code>	Remove display blob and original, if any
GET	<code>/uploads/photos/:tripId/:photoId</code>	Serve display JPEG (signed query when locked)

A scheduled function expires photos older than 365 days and destroys any linked Cloudinary original.

JSON bodies are capped at 200 KiB. Display uploads are capped at 4 MiB and 100 photos per trip; the client compresses to a longest edge of 1600 pixels. Originals, when enabled, allow JPEG, PNG, and HEIC/HEIF up to 15 MiB, stored as Cloudinary *authenticated* assets. Signed original URLs last one hour. Display URLs for locked trips are HMAC-signed and last one hour. Photo session tokens last 30 days.

Creating a trip requires `PHOTO_PIN_PEPPER` in the environment. Cloudinary keys are optional: without them, gallery upload still stores the compressed JPEG and `originalUrl` remains empty.

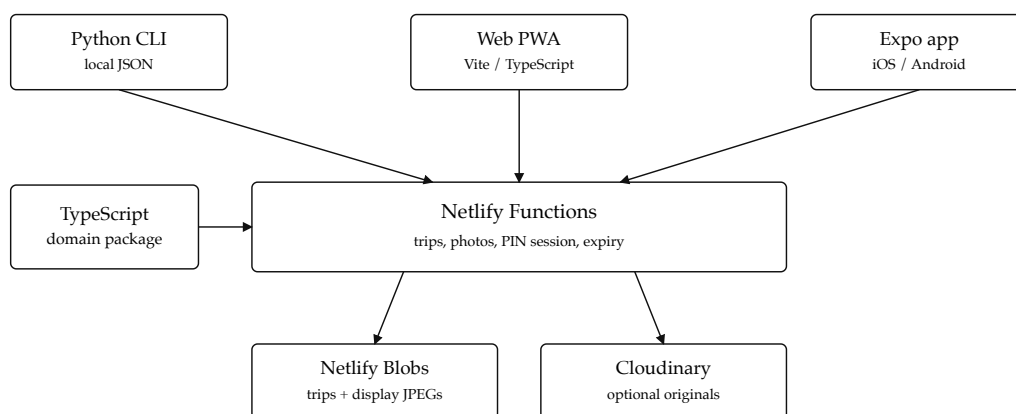


Figure 1. Hosted data path. The CLI also reads and writes a local trip file; `pull/push` are the only CLI calls into the Functions. The TypeScript domain package is the money and PIN implementation for Functions, web, and (via HTTP) mobile.

7 Trust and security model

There are no accounts. Possession of the 32-character trip id is permission to read and rewrite people and expenses. The id is unguessable in ordinary use; it is not a secret against someone who already has the link. That is intentional: a shared trip is meant to be edited by everyone on the holiday.

Photos are the exception. A six-digit PIN is generated at trip creation, hashed with SHA-256 over `pepper:tripId:pin`, and stored beside the trip. The pepper is a server secret (`PHOTO_PIN_PEPPER`). Unlocking Moments issues an HMAC session token bound to the trip. PIN guesses are rate-limited per IP. Trips created before PIN hashing existed can set a PIN once (`POST .../pin`); until then, the gallery behaves as unlocked.

The PIN is shown to whoever creates the trip (and can be copied from the trip page). It is not in the downloadable JSON. Losing it means losing gallery administration on a locked trip; expenses remain editable via the link.

Secrets do not belong in the repository. Local Functions read `.env`; production reads the Netlify UI. `.env` is gitignored; `.env.example` lists names only.

8 Capabilities, by concern

Table 4. What each surface can do.

Capability	CLI	Web	Mobile
Create a local trip file	yes	—	—
Create a hosted trip	via push	yes	yes
Add / edit / remove expenses	yes	yes	yes
Equal and weighted splits	yes	yes	yes
Balances and settlement	yes	yes	yes
Share by URL	after push	yes	yes
Import / export money JSON	pull/push	yes	—
Photo gallery (Moments)	—	yes	yes
Set PIN on a legacy trip	—	yes	yes
Optional Cloudinary original	—	yes	yes
Offline-first local file	yes	—	—

“Import / export money JSON” on the web is Download JSON on the trip page and Open JSON on the home page. The CLI equivalents are `pull` (into the current file) and `push` (always a new hosted trip). Photos never travel in that file.

9 Development, tests, and deployment

Continuous integration runs on GitHub Actions for `main` and `pull` requests:

- Python 3.11–3.13: Ruff, then Pytest with an 80% coverage floor on `fairshare`.
- Web: Vitest, then a production Vite build.
- Mobile: Vitest and `tsc --noEmit`.

The site deploys from the linked Netlify project. A production CLI deploy builds Functions and `web/dist` and publishes to <https://fair-share-trips.netlify.app>.

Store submission is outside the Netlify path. It needs an Expo account, `eas-cli` login and init, and App Store Connect / Play Console accounts. The first review is expected to keep the donation URL empty if the recipient is the developer rather than a registered nonprofit.

10 Limits and non-goals

- Concurrent PUT of a trip is last-write-wins. There is no operational transform and no per-field merge.
- Settlement is heuristic, not a proven minimum transfer set.
- Setting a PIN on a grandfathered trip re-reads before write to shrink the race; it is not a compare-and-swap.

- Display photos are compressed; the original is optional infrastructure, not a guarantee.
- The CLI money engine is Python; the hosted engine is TypeScript. They are kept in agreement by tests and review, not by a single compiled artefact.
- The product is not a bank, not a receipt archive, and not an identity provider.

11 Licence

The software is released under the MIT License (copyright 2026). See `LICENSE` in the repository root.

12 Summary

`FAIR SHARE` is a trip splitter whose money path never leaves the integers, whose hosted collaboration is a secret URL rather than an account system, and whose photos sit behind a PIN and a retention clock. The repository packages that idea as a CLI for files, a PWA and an Expo app for shared links, and a small Netlify API in front of Blobs and optional Cloudinary. The capabilities in Section 5 and Table 4 are the product; everything else is scaffolding to keep a cent from disappearing.